The Problem with Open Classes

You and Bill don't have to look much further before you stumble upon another opportunity to use Open Classes. The Bookworm source contains a method that replaces elements in an array:

object_model/replace.rb

```
def replace(array, original, replacement)
  array.map {|e| e == original ? replacement : e }
end
```

Instead of focusing on the internal workings of replace, you can look at Bookworm's unit tests to see how that method is supposed to be used:

```
def test_replace
  original = ['one', 'two', 'one', 'three']
  replaced = replace(original, 'one', 'zero')
  assert_equal ['zero', 'two', 'zero', 'three'], replaced
end
```

This time, you know what to do. You grab the keyboard (taking advantage of Bill's slower reflexes) and move the method to the Array class:

```
class Array
  def replace(original, replacement)
    self.map {|e| e == original ? replacement : e }
  end
end
```

Then you change all calls to replace into calls to Array#replace. For example, the test becomes as follows:

```
def test_replace
  original = ['one', 'two', 'one', 'three']
➤ replaced = original.replace('one', 'zero')
  assert_equal ['zero', 'two', 'zero', 'three'], replaced
end
```

You save the test, you run Bookworm's unit tests suite, and...whoops! While test_replace does pass, other tests unexpectedly fail. To make things more perplexing, the failing tests seem to have nothing to do with the code you just edited. What gives?

"I think I know what happened," Bill says. He fires up irb, the interactive Ruby interpreter, and gets a list of all methods in Ruby's standard Array that begin with *re*:

```
[].methods.grep /^re/  # => [:reverse_each, :reverse, ..., :replace, ...]
```